

LocalRCA: On-Device Root Cause Analysis for Windows Endpoints Without Ground Truth

Yashash Mathur

*Dept. of Information Science and Engineering
RV Institute of Technology and Management
Bengaluru, India
yashashgdg@gmail.com*

Aditya Prakash

*Dept. of Information Science and Engineering
RV Institute of Technology and Management
Bengaluru, India*

Utsav Upadhyay

*Dept. of Information Science and Engineering
RV Institute of Technology and Management
Bengaluru, India*

Shresth Modi

*Dept. of Information Science and Engineering
RV Institute of Technology and Management
Bengaluru, India*

Dr. Niharika P Kumar

*Associate Professor, Dept. of ISE
RV Institute of Technology and Management
Bengaluru, India*

Abstract—Post-hoc diagnosis of a personal or workstation-class machine is poorly served by existing tooling: cloud observability stacks assume a fleet, a network path and a willingness to export telemetry, while the Windows Event Log records that a fault occurred but rarely why. We present LocalRCA, a desktop system that records system telemetry on a single machine, learns that machine’s normal behaviour with an LSTM autoencoder, and attempts to explain incidents using Granger causality constrained by false-discovery-rate correction, an effect-size floor and a subsystem topology prior. The system runs entirely on the endpoint; the collector opens no sockets, and the only network capability is an opt-in, off-by-default check for a newer release that transmits nothing. Because a personal machine offers no labelled incidents, we evaluate in two ways: seven controlled fault injections, which supply ground truth by construction, and a population survey that replays the pipeline over 596 incidents discovered in real collected history. The system detects and correctly attributes injected CPU, disk and memory faults, and in one case produced a correct causal explanation of a fault it was never told about, with six surviving edges oriented away from the injected cause. Across real history it produces a supported causal chain for 106 of 596 incidents (17.8%), with 36% falling below the Granger sample floor and therefore untestable at the default lag. We further show that the hand-written subsystem prior rejects 22.7% of statistically accepted pairs and forbids mechanisms that demonstrably exist. Deploying the system to non-specialist users imposed two further requirements, which we describe with their limits: the learned model is encrypted at rest under the user’s Windows credentials, and the interface leads with one plain-language verdict per screen while keeping every raw measurement one switch away. We argue that the most useful property of such a system is its willingness to report that it cannot explain an incident, and we quantify how often that occurs.

Index Terms—root cause analysis, anomaly detection, LSTM autoencoder, Granger causality, endpoint telemetry, false discovery rate, explainability

I. INTRODUCTION

Diagnosing why a personal computer became slow, stalled or crashed is typically a manual exercise: an engineer lines up resource graphs against the Windows Event Log and guesses. Fleet-oriented observability platforms automate this, but assume infrastructure a single machine does not have—a collector network, a time-series backend, and consent to export telemetry describing which applications a person runs.

Three constraints follow from targeting a single endpoint, and they shape the entire design.

No egress. Telemetry describing application usage is sensitive. The system is built so that nothing is transmitted; the privacy property is structural, enforced by the absence of network code, rather than by policy.

No ground truth. A personal machine has no labelled incidents. The model cannot be trained to recognise known faults and cannot be evaluated against a labelled test set. This is the most consequential constraint, and Section V describes the only workable response we found: manufacture ground truth by causing faults deliberately.

Cold start is unavoidable. “Normal” must be learned from the machine itself, so the tool is useless until it has observed enough normal behaviour—approximately 21 hours of clean collection in our configuration.

Our contributions are:

- 1) An end-to-end on-device RCA pipeline combining reconstruction-error anomaly detection with statistically constrained Granger causality (Section IV).
- 2) A fault-injection harness that manufactures ground truth on a machine that has none, and a demonstration that

the pipeline can identify a cause it was not told about (Section V).

- 3) A population-scale measurement of how often the causal layer produces an answer at all, replacing claims previously resting on single runs (Section VI).
- 4) An audit showing that the hand-written subsystem prior—not the statistical gates—is the largest single filter in the pipeline, and that it forbids physically real mechanisms (Section VII).
- 5) A deployment design for a tool that holds sensitive telemetry—per-user encryption of the learned model, signed binaries, and an interface that separates the verdict from the evidence—together with the places it falls short (Section VIII).

II. RELATED WORK

Anomaly detection in multivariate time series. Reconstruction-based detectors using LSTM autoencoders are a standard approach when labelled anomalies are unavailable [1], [2]. Training on nominal data only, and thresholding reconstruction error, avoids requiring labelled faults—which is precisely the constraint an endpoint imposes.

Causal discovery from observational data. Granger causality [3] tests whether one series improves prediction of another beyond the second’s own history. It is correlational under a temporal constraint rather than interventional, and is well known to over-report on correlated series without correction. We apply Benjamini–Hochberg false-discovery-rate control [4] across the full set of ordered pairs, an effect-size floor, and a domain prior over subsystems.

Root cause analysis in distributed systems. Microservice RCA work typically exploits service call graphs or distributed traces to constrain candidate causes [5], [6]. No such topology exists on a single endpoint, which motivates our explicit, inspectable subsystem map—and, as Section VII shows, makes the quality of that map a first-order concern.

Benchmarked causal RCA at fleet scale. Recent work evaluates causal discovery for root cause analysis at a scale this system does not attempt. Ikram et al. [7] discover root causes across thousands of microservice nodes and report top- k recall against ground truth gathered from a real production incident corpus. Pham et al. [8] benchmark six causal-RCA methods (including CausalRCA and BARO) across microservice architectures and find that none dominates uniformly, that performance is highly sensitive to the underlying call graph, and that published recall numbers do not transfer cleanly between systems. Both works assume a call graph or trace corpus that a single endpoint does not have; the corresponding gap here is that our 596-incident survey has no comparable ground truth, so Section VI reports how often the system speaks, not how often it is correct against a labelled corpus.

Positioning. The distinguishing constraints of this work are the single-machine scope, the absence of ground truth, and a deliberate design commitment to reporting uncertainty rather than producing a ranking regardless of evidence.

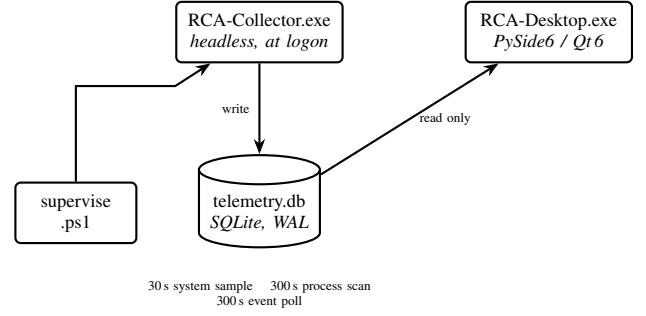


Fig. 1. Process architecture. The collector writes; the desktop application only reads. A PowerShell supervisor restarts the collector if it exits unexpectedly, because creating a Task Scheduler logon task requires elevation that a diagnostic tool should not demand.

III. SYSTEM ARCHITECTURE

The system comprises two processes sharing a SQLite database (Fig. 1). The separation exists because collection must survive the user interface being closed.

The collector samples 29 features every 30 seconds, the top processes by CPU and resident memory every 300 seconds (tightening to 30 seconds under load), and an allow-listed subset of Windows Event Log records. Retention is tiered by sensitivity: numeric readings for 365 days, per-process detail for 30 days, and the foreground application name—the most personal field collected—for 30 days.

At the time of writing the evaluation host had accumulated 124,674 system samples, 1,835,250 process samples and 6,600 Event Log records over 59 days, occupying 227 MB. Coverage of that span is 73.1%: 157 breaks in collection account for 383 hours not recorded. The population survey (Section VI) draws on the most recent 30 days of this history.

IV. METHOD

A. Scaling and windowing

A min–max scaler is fitted on the clean baseline only, $x'_j = (x_j - \min_j) / (\max_j - \min_j)$, and incident windows are transformed with the baseline’s parameters and clipped to $[0, 1]$. Fitting on the incident would normalise away the deviation being sought.

Training windows are built *within contiguous segments only*, never spanning a collection gap. A segment is a maximal run of samples whose spacing is within tolerance of the sampling cadence.

B. Anomaly detection

An LSTM autoencoder compresses a window of T samples over $n = 29$ features to a 32-dimensional latent vector and reconstructs it. Only the encoder’s final hidden state reaches the bottleneck, so the entire window is represented by 32 numbers before reconstruction; that compression is the detection mechanism. Training minimises

$$\mathcal{L} = \frac{1}{BTn} \sum_{b,t,j} (x_{btj} - \hat{x}_{btj})^2. \quad (1)$$

Error is reduced over time but *not* over features, which is what makes the output attributable to a metric rather than to a window:

$$s_j = \frac{1}{T} \sum_{t=1}^T (x_{tj} - \hat{x}_{tj})^2. \quad (2)$$

Per-metric thresholds are the 99th percentile of validation error, $\tau_j = P_{99}\{s_j(w)\}$, and a metric is anomalous when $s_j > \tau_j$. Per-metric thresholds are necessary because reconstructability varies widely: a near-constant channel reconstructs almost perfectly, while a bursty one does not, and a single global threshold would flag the latter permanently.

C. Constrained causal inference

For each ordered pair of anomalous metrics, both series are differenced until an Augmented Dickey–Fuller test [10] rejects a unit root (at most twice), then tested for Granger causality at lags $1 \dots L$. The lag minimising the p -value is selected. A pair is skipped unless

$$N \geq 3L + 2, \quad (3)$$

which at the default $L = 5$ requires 17 aligned samples. Equation (3) explains the majority of this system’s negative results.

Surviving p -values are corrected with Benjamini–Hochberg: sorting ascending, the largest rank k satisfying $p_{(k)} \leq \alpha k/m$ is found and ranks $1 \dots k$ accepted, with $\alpha = 0.05$. A p -value alone is insufficient, since negligible improvements become significant given a long history, so an effect-size proxy is also required:

$$\text{effect} = \frac{F}{F + N} \geq 0.10. \quad (4)$$

This quantity is monotone in F , bounded in $(0, 1)$, and shrinks as N grows — which is the correction desired, since large N is what made the p -value small.

Accepted pairs form a directed graph; cycles are removed preferring edges that contradict temporal precedence, and remaining edges are filtered against a subsystem topology prior (Section VII). Candidates are then ranked by a weighted composite plus PageRank [9] on the reversed graph:

$$\text{score}(v) = \min\left(1, 0.70 \sum_k w_k s_k(v) + 0.30 \text{PR}(v)\right), \quad (5)$$

with weights 0.40 causal outflow, 0.30 temporal priority, 0.20 low inflow, 0.05 severity and 0.05 event correlation.

A structural caveat. With zero surviving edges, every node has outflow 0 and inflow 1, so 60% of the weight becomes constant and the ranking collapses onto severity and timing while still emitting a score. The system therefore reports the distinction explicitly rather than presenting the ranking as causal.

V. EVALUATION BY FAULT INJECTION

Because a personal machine provides no labelled incidents, we manufacture ground truth: a harness causes a specific, known disturbance, waits for samples to land at the real cadence, runs the production pipeline over the injection window, and scores the result.

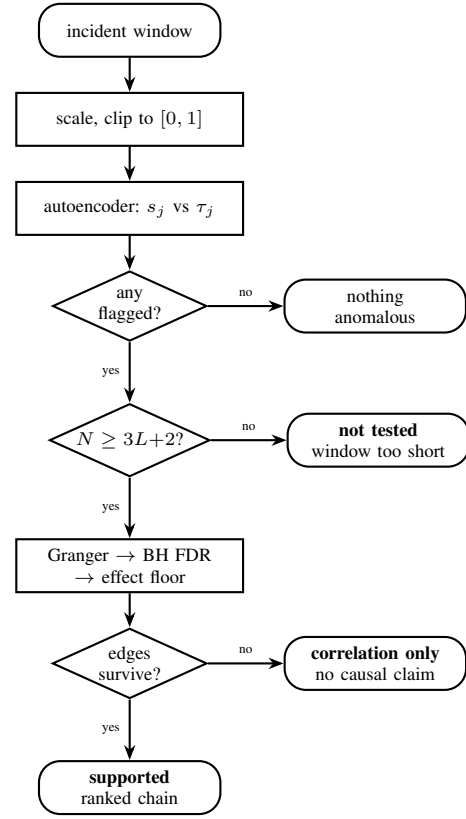


Fig. 2. Inference pipeline. Four terminal states are reported distinctly. Collapsing “not tested” into “nothing survived” would conflate the absence of a result with a negative result, which earlier revisions of this system did.

TABLE I
CONTROLLED FAULT INJECTIONS, 30-MINUTE WINDOWS UNLESS NOTED

Fault	Samples	Flagged	Edges	Outcome
CPU, 7 min	14	6/29	—	not tested (below floor)
CPU	60	6/29	6	explained correctly
Disk	60	4/29	0	detected, unexplained
Memory	60	2/29	0	wrong process named
Memory (refixed)	60	3/29	0	detected, attributed
Memory (clean)	60	0/29	—	not measurable on host
Idle	60	1/29	—	3.4% false positives

A. A correct explanation of an unknown cause

Half the CPU cores were burned for 30 minutes with no information given to the pipeline. The ranking named `cpu_pct` first at 1.000, and the surviving edges are oriented *away* from CPU—the signature of a genuine root cause, since nothing upstream drives it:

$$\text{cpu_pct_max_core} \rightarrow \text{disk_busy_pct} \\ (\text{lag } 2, 0.756).$$

B. Starvation, not failure

The same fault at 7 and at 30 minutes differed only in duration. At 14 samples, against the floor of 17 from (3), *no pair was compared at all*, and the system reported exactly that. At 60 samples, ten pairs survived correction and six survived

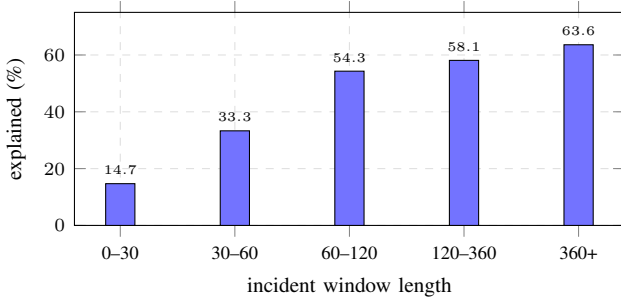


Fig. 3. Causal yield against window length, over 381 analysable incidents ($L = 5$, 30-day survey). The rate rises monotonically with window length, confirming at scale the starvation effect first seen in a single pair of injections.

the topology filter. The causal layer was not broken; it was starved. This distinction changes the remedy from redesign to window width, and is confirmed at scale in Section VI.

C. A defect that injection exposed

The memory injection detected the anomaly and then attributed it to four unrelated processes. Attribution ranked candidates by mean CPU, while resident memory was selected but never sorted on—so a process that allocates and sleeps could never be named. A memory-bound cause was unreportable *by construction*, in every incident the system had ever analysed. The CPU and disk faults are both CPU-heavy and had attributed correctly by accident of their shape, concealing the defect.

D. A measurement this host cannot provide

A later memory injection on a deliberately cleared machine flagged nothing. The detector was correct: the host’s median memory utilisation across 30,199 samples is 95.2%, and the injection window sat at 87.4%—the 9th percentile of its own history. The injection is bounded to half of free memory for safety, which on a habitually-full host cannot exceed the level the machine already reaches unaided. The fault is undetectable *by construction* here, and the harness now reports this as inconclusive rather than as a detector failure.

VI. POPULATION-SCALE EVALUATION

Fault injection establishes correctness on a handful of cases but yields single observations. To measure *frequency*, the pipeline was replayed over every incident discoverable in real collected history—no injection, read-only.

Of 596 incidents discovered, 215 (36%) fall below the sample floor of (3) and cannot be tested at any setting. Of the 381 that can be, 106 produce a supported causal chain. The end-to-end figure a user experiences is therefore $106/596 = 17.8\%$: roughly one incident in six receives a causal explanation.

We report the funnel rather than the survivor-filtered rate. The share of *analysable* incidents explained is 27.8%, which is the more flattering number and describes fewer situations.

The floor is a parameter artefact. Short incidents are widened before analysis to exactly the model’s window size,

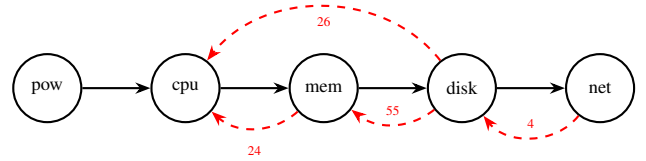


Fig. 4. The subsystem prior is a strict total order (solid), so *network* is a sink and nothing can be reported as caused by network activity. Dashed edges are directions the statistics accepted and the map rejected, labelled with occurrence counts (30-day, 596-incident survey). Two of them describe real mechanisms: the file cache consuming memory under write pressure, and interrupt handling costing CPU during disk activity; a download writing to disk (*network*→*disk*) remains a smaller but real third case.

15 samples, while (3) at $L = 5$ requires 17—two constants chosen independently in the same pipeline. Re-running the survey at $L \in \{3, 4, 5\}$ over an identical population (598 incidents discovered over 30 days of real collected history) confirms the cliff disappears at $L \leq 3$: 0 incidents fall below the floor, versus 215 at $L = 5$. Explanations rise only modestly, from 106 to 110, while the tested population nearly triples (381 to 598), and median surviving edges holds at 2 throughout. The gain is honest reporting—“tested, nothing found” instead of “not tested”—rather than insight.

VII. AUDITING THE SUBSYSTEM PRIOR

Since no service call graph exists on an endpoint, the system encodes domain knowledge as an explicit map of permissible subsystem influence. Auditing it across all analysable incidents shows it is the largest single filter in the pipeline after the statistical gates: of 538 accepted pairs, 337 (62.6%) survive, 122 (22.7%) are rejected by the map and 79 (14.7%) by cycle-breaking.

Every rejected transition has *zero* survivors, because the map forbids whole classes rather than individual edges (Fig. 4). Judged individually, *disk*→*memory* is almost certainly real—the Windows standby file cache expanding under write pressure—and is the strongest single rejected relationship measured in this run, at 0.654 (*disk_write_bps*→*swap_used_delta*, lag 2). *disk*→*cpu* reflects interrupt and DPC handling, and *network*→*disk* is the download-writes-to-disk case argued in the prior revision of this analysis, present but smaller in this run (4 occurrences). Conversely, the smallest rejected classes (*network*→*process*, *disk*→*process*, at 1–2 occurrences each) are plausibly spurious rather than real mechanisms being wrongly forbidden.

The prior is therefore not so much wrong as *one-directional*: it models resource pressure flowing downhill and has no vocabulary for the feedback that makes I/O expensive. Adding the reverse edges would make the graph cyclic, where cycle-breaking already removes 14.7% of pairs—so the two mechanisms would begin to compete. We record this as an open design question rather than patching it.

VIII. DEPLOYMENT ENGINEERING

A research prototype can leave its data in plaintext and its results as raw tables. A tool installed by someone else, holding a record of when and how their machine is used, cannot. This section describes what the released application (v1.5.1) does about both, and where it falls short.

A. Protecting the model at rest

The trained artefact holds the scaler’s per-metric bounds and the per-metric thresholds—a compact statistical profile of how the machine is used. It is wrapped with the Windows Data Protection API (DPAPI), which binds the ciphertext to the user’s logon credentials, so a copy taken to another account or machine cannot be read. The write is atomic, via a temporary file and a rename, so an interrupted save cannot leave a truncated model. Artefacts from earlier versions carry no header, still load, and are encrypted the next time the model is trained. If the user’s credentials are reset and the key is lost, the application says the model must be retrained rather than surfacing a raw Windows error. The database and logs, which DPAPI does not wrap, are covered by a second, best-effort layer: at start-up the collector asks the Encrypting File System (EFS) to encrypt its data folder.

Three findings qualify this. First, EFS does not exist on Windows Home editions. The evaluation host runs one, the request fails on every start, and its 227 MB database remains in plaintext, protected only by file-system permissions. The failure is logged rather than silent, but the protection a user receives depends on which edition of Windows they bought. Second, re-encryption happens only on retraining, so the model on the evaluation host—trained before encryption was added—is still the plaintext artefact. Third, independent review of the encryption change found that training wrote an intermediate checkpoint in plaintext and deleted it only on success; an interrupted run left unencrypted weights on disk. The checkpoint is now removed in a *finally* block, and any leftover from an earlier crash is purged when training starts.

B. Signed binaries

Both executables are Authenticode-signed with an RFC 3161 timestamp, and each release publishes a SHA-256 checksum. The certificate is self-signed: it shows that a binary is unchanged since it was built, not who built it, so Windows SmartScreen still warns on first run. Removing the warning requires a certificate from an authority Windows trusts, which this project does not have.

C. Separating the verdict from the evidence

A review of the running application found that the first screen presented a table of every captured channel with its raw latest value, and that training readiness was expressed four different ways (clean samples, longest run, current run, time remaining) that a non-specialist had to reconcile alone. Hiding these would contradict the project’s premise, since a user cannot consent meaningfully to collection they cannot inspect.

The released interface therefore has two presentations of identical computation, selected by a single global switch that persists across sessions and is off by default. In the default view each tab leads with one sentence whose colour encodes state: the data tab reports *Healthy* only when coverage is at least 90% with at most three breaks, and *Partial data* from 50%; the training tab states whether a model can be trained and, if not, how long remains. The inference tab keeps the incident picker and opens on the plain-language report rather than the score table. The advanced view restores the raw counts, the channel table, the database path, the training hyperparameters, the Granger lag, and the score and outflow columns. Nothing is computed differently in either view; the switch changes presentation only, and the headline sentence is derived from the same coverage figures the advanced view prints. The evaluation host itself reads *Partial data* at 73% coverage—accurate, if unflattering.

This design was checked by one expert review and by driving the live application. It has not been measured with users, and whether the default view helps a non-specialist reach a correct conclusion is unestablished.

IX. DISCUSSION AND THREATS TO VALIDITY

All measurements come from one machine. Nothing here establishes generality, and one measurement—memory-fault detection—is provably unobtainable on this host, which makes second hardware a prerequisite rather than an enhancement.

Correctness is not established at population scale. The survey’s 596 incidents have no known cause, so a 17.8% explanation rate reports how often the layer speaks, never whether it is right. The rate is a ceiling on usefulness, not a measure of accuracy.

No fault has been tested whose cause is not also the loudest metric. Both explaining runs put the injected fault atop a severity ranking, so a correct causal answer and a correct severity answer remain indistinguishable.

Granger causality is not causation. It is correlation under a temporal constraint. The FDR correction, effect floor and subsystem prior exist to suppress the resulting over-reporting, and Section VII shows the prior itself is imperfect.

A methodological observation. Nine defects were found in code carrying a passing test suite of 130 tests. In nearly every case the test asked a question *adjacent* to the one that mattered—a database value instead of the bytes on disk, one file instead of the full storage footprint, a function instead of the code path calling it. Two fixes were worse than the defects they repaired. These were caught by adversarial review rather than additional testing, which we believe generalises beyond this system. The pattern recurred once encryption was added: two reviewers independently found the plaintext training checkpoint of Section VIII in code whose tests all passed, because the tests exercised only the successful path. The suite now stands at 150 tests.

X. CONCLUSION

We presented an on-device root cause analysis system for Windows endpoints operating without ground truth, network

access or fleet context. Under controlled injection it detects and attributes CPU, disk and memory faults, and has produced a correct causal explanation of a fault it was never told about. Across real history it explains roughly one incident in six, declines to explain the rest, and distinguishes “not tested” from “tested and nothing survived”.

We argue this last property matters more than the explanation rate. A tool reporting “no causal chain was supported” when the data cannot support one is more useful than a confident ranking that changes with the analysis window—and in this system, the engineering required to make it say so honestly exceeded the engineering required to compute an answer at all.

Future work: repeating the evaluation on second hardware, testing a fault whose cause is not the loudest metric, resolving whether the subsystem prior should admit the feedback directions it currently forbids, encrypting the database independently of EFS so that Home-edition users receive the same protection, and measuring with users whether the default view leads them to correct conclusions.

REFERENCES

- [1] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based encoder-decoder for multi-sensor anomaly detection,” in *Proc. ICML Anomaly Detection Workshop*, 2016.
- [2] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding,” in *Proc. ACM SIGKDD*, 2018, pp. 387–395.
- [3] C. W. J. Granger, “Investigating causal relations by econometric models and cross-spectral methods,” *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.
- [4] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: a practical and powerful approach to multiple testing,” *J. Roy. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
- [5] M. Kim, R. Sumbaly, and S. Shah, “Root cause detection in a service-oriented architecture,” in *Proc. ACM SIGMETRICS*, 2013, pp. 93–104.
- [6] P. Chen, Y. Qi, P. Zheng, and D. Hou, “CauseInfer: automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems,” in *Proc. IEEE INFOCOM*, 2014, pp. 1887–1895.
- [7] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, “Root cause analysis of failures in microservices through causal discovery,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 31 158–31 170.
- [8] L. Pham, H. Ha, and H. Zhang, “Root cause analysis for microservice system based on causal inference: How far are we?” in *Proc. 39th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024.
- [9] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: bringing order to the web,” Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [10] D. A. Dickey and W. A. Fuller, “Distribution of the estimators for autoregressive time series with a unit root,” *J. Amer. Statist. Assoc.*, vol. 74, no. 366, pp. 427–431, 1979.